SOFTWARE REQUIREMENTS SPECIFICATION

YOYOL School Management System (ERP)

An Open-Source School ERP

Prepared by NOTTECH Company Limited for internal development

Document Version: 0.4 (Draft)

Date: 7 August 2026

License: MIT | Repository: Public (GitHub)

Table of Contents

Table of Contents 2
Document Control 4
Revision History 4
Confirmed Engineering Decisions 4
Nature of This Document 4
1. Introduction 5
1.1 Purpose 5
1.2 Scope 5
1.3 Definitions, Acronyms and Abbreviations 5
1.4 Intended Audience 5
1.5 References 5
2. Overall Description 7
2.1 Product Perspective 7
2.2 Product Functions (Module Summary) 7
2.3 User Classes and Characteristics 7
2.4 Operating Environment 7
2.5 Assumptions and Dependencies 7
3. Functional Requirements 8
3.1 Identity & User Management 8
3.2 Student & Guardian Management 8
3.3 Staff Management 8
3.4 Academic Structure 8
3.5 Timetables & Lesson Plans 8
3.6 Assessments & Results 9
3.7 Finance & Payments 9
3.8 Attendance 9
3.9 Discipline, Clubs & Activities 9
3.10 Calendar & Student Leadership 9
4. Data Model 10
4.1 Master Tables 10
4.2 Transactional Tables 10
4.3 Join Tables 10
5. System Architecture 12
5.1 Architectural Overview 12
5.2 Layered Structure 12
5.3 Request Flow 12
5.4 Runtime Portability Note 12
6. Technology Stack & Database Adapter 13
6.1 Technology Stack 13
6.2 Database Adapter Design 13
6.2.1 Drizzle Driver per Target 13
6.2.2 Configuration 13
6.2.3 Adapter Contract (illustrative) 13
6.2.4 Migrations 14
6.2.5 Adapter Requirements 14
6.3 Database Layer Decision (Drizzle ORM) 14
7. Development Methodology & Testing 15

7.1 Test-Driven Development 15
7.2 Testing Framework (Specified) 15
7.3 Test Levels 15
7.4 Testing Requirements 15
8. Documentation Requirements 16
8.1 API Documentation 16
8.2 System Documentation 16
8.3 Contributor & User Documentation 16
9. Coding Standards & Conventions 17
9.1 Language & Style 17
9.2 Version Control & Reviews 17
9.3 Quality Gates in CI 17
10. Open-Source Governance 18
10.1 Licensing & Repository Hygiene 18
10.2 Contribution & Release Process 18
11. DevOps, Containerisation & Deployment 19
11.1 Containerisation 19
11.2 CI/CD 19
11.3 Deployment Targets 19
12. Security 20
12.1 Authentication 20
12.2 Authorization 20
12.3 Application & Supply-Chain Security 20
13. Data Protection & Privacy 21
13.1 Data Classification 21
13.2 Requirements 21
14. Observability & Error Handling 22
14.1 Logging & Monitoring 22
14.2 Error Handling 22
14.3 Audit Trail 22
15. API Design & Conventions 23
16. Accessibility & Internationalization 24
16.1 Accessibility 24
16.2 Internationalization & Localization 24
17. Non-Functional Requirements 25
18. Planned Feature Set (Roadmap) 26
18.1 Primary: Mobile Money Fee Collection & Automated Reconciliation 26
Why it fits YOYOL 26
Outline capabilities 26
18.2 Secondary Roadmap Items 26
19. Open Items for Expansion 27

Document Control

Revision History

Version	Date	Author	Summary of Changes
0.1	7 Aug 2026	NOTTECH	Initial draft: scope, functional requirements, and data model.
0.2	7 Aug 2026	NOTTECH	Added architecture, database adapter, technology stack, TDD strategy, documentation plan, coding standards, open-source governance, DevOps, security, and a proposed feature set.
0.3	7 Aug 2026	NOTTECH	Finalised the database layer decision (Drizzle ORM) as an internal requirement; specified the testing framework (Vitest with the Workers pool, plus Playwright). Reframed as an internal NOTTECH development document.
0.4	7 Aug 2026	NOTTECH	Added dedicated Security, Data Protection & Privacy, Observability & Error Handling, API Design, and Accessibility & Internationalization sections (JWT auth specified) to reach world-class coverage.

Confirmed Engineering Decisions

The following decisions are fixed inputs for the build:

- Open-source license: MIT.
- Monorepo and package manager: pnpm workspaces.
- Frontend and backend framework: React Router 7 (framework mode, full-stack).
- Database layer: a configurable adapter supporting both Cloudflare D1 and PostgreSQL, implemented with Drizzle ORM (see Section 6).
- Testing framework: Vitest, using the Cloudflare Workers pool for the edge/D1 path, with Playwright for end-to-end tests (see Section 7).
- Delivery: fully dockerized; open-source project hosted publicly on GitHub.
- Engineering practices: T from the outset; API documentation and system documentation required; a defined coding standard required.

Nature of This Document

This SRS is prepared by NOTTECH Company Limited and is used internally by NOTTECH to build the YOYOL platform. It is the authoritative engineering specification for the project team. The technical decisions recorded here are internal engineering decisions, not proposals awaiting external sign-off; where a choice was previously left open, it has now been made and is stated as a requirement.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional, architectural, and engineering-process requirements for the YOYOL School Management System, an open-source Enterprise Resource Planning (ERP) platform for schools. It provides a shared baseline for contributors, maintainers, and stakeholders, and governs how the system is designed, built, tested, documented, and released. This is a draft intended for continued expansion.

1.2 Scope

YOYOL centralises the administrative and academic operations of a school in a single system: identity and access, student and staff records, academic structures, assessments and results, finance and fee collection, attendance with parent notification, timetables, lesson plans, discipline, clubs and activities, the school calendar, and student leadership. It supports multiple academic years and role-based access.

As an open-source project, YOYOL additionally targets self-hostability and portability: it runs on Cloudflare's edge (Workers with D1) or on a conventional server with PostgreSQL, selected purely through configuration. The project is optimised for the East African education context, with mobile-first access and low-bandwidth tolerance.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification.
ERP	Enterprise Resource Planning.
RBAC	Role-Based Access Control.
TDD	Test-Driven Development: write a failing test, make it pass, then refactor.
SSR	Server-Side Rendering.
Adapter	A pluggable module implementing a fixed contract; here, the database adapter for D1 or Postgres.
D1	Cloudflare's serverless SQLite-compatible database.
ORM	Object-Relational Mapper / query builder abstracting SQL access.
CI/CD	Continuous Integration / Continuous Delivery.
ADR	Architecture Decision Record.
OpenAPI	A specification standard for describing HTTP APIs.

1.4 Intended Audience

- The NOTTECH engineering team building YOYOL (primary audience).
- NOTTECH project leads, for scope and delivery tracking.
- Future open-source contributors, once the project is published.
- Self-hosters and system administrators deploying YOYOL.

1.5 References

- YOYOL initial design and data-model notes (source discussion).
- React Router 7 framework-mode and Cloudflare Workers deployment documentation.
- IEEE 830 (SRS structure), Semantic Versioning, Keep a Changelog, Conventional Commits, Contributor Covenant.

2. Overall Description

2.1 Product Perspective

YOYOL is a self-contained, full-stack web application. The frontend and backend are unified under React Router 7 in framework mode, with server-side rendering and server-side data loaders and actions. Persistence is reached only through a database adapter, so the same application code runs unchanged against Cloudflare D1 or PostgreSQL. The entire system is containerised for reproducible local development and self-hosting.

2.2 Product Functions (Module Summary)

Module	Core Function
Identity & Users	School profile, user accounts, roles, and access control.
Student & Guardian	Student registration and linked parent/guardian details.
Staff	Teacher and administrative staff records.
Academics	Subjects, levels, streams, classrooms, combinations, timetables, lesson plans.
Assessment & Results	Assessment types and recording of marks, grades, and comments.
Finance	Payment terms, fee structures, and payment tracking.
Attendance	Daily attendance with optional parent notification.
Behaviour & Activities	Discipline records, clubs, and club activities.
School Life	Calendar/events and student leadership positions.

2.3 User Classes and Characteristics

Role	Description & Key Capabilities
Admin	Full configuration: branding, users, academic structures, and master data.
Teacher	Records results, attendance, lesson plans, and discipline for assigned classes/subjects.
Finance	Manages payment terms, fee structures, and records payments.
Student	Views personal results, attendance, timetable, calendar, and leadership status.
Parent / Guardian	Receives notifications and views a linked student's records where permitted.

2.4 Operating Environment

- Client: modern desktop and mobile browsers; mobile-first, low-bandwidth tolerant.
- Server (Option A): Cloudflare Workers runtime with D1 as the database.
- Server (Option B): Node.js container with PostgreSQL as the database.
- Local development: Docker Compose bringing up the app and a Postgres instance.

2.5 Assumptions and Dependencies

- Each user account maps to exactly one role; a user links to at most one staff or student record.
- A student may have multiple parents/guardians (one-to-many).
- Fee structures are defined per class and payment term.
- The database adapter contract is identical across D1 and Postgres; only configuration differs.
- Cloudflare Workers is not a Node.js runtime; server code must remain runtime-portable behind the adapter and framework abstractions.

3. Functional Requirements

Requirements are grouped by module with unique IDs for traceability. This draft captures baseline behaviour; detailed rules and validations will be added during expansion.

3.1 Identity & User Management

ID	Requirement
FR-1.1	The system shall store a school profile including name, motto, logo, theme colours, address, phone, and email.
FR-1.2	The system shall support user accounts with a username and securely hashed password.
FR-1.3	The system shall assign each user exactly one role (Admin, Teacher, Finance, Student, Parent).
FR-1.4	The system shall enforce role-based access control across all modules.
FR-1.5	The system shall optionally link a user account to a staff or student record.

3.2 Student & Guardian Management

ID	Requirement
FR-2.1	The system shall register students with names, gender, date of birth, admission number, and date of admission.
FR-2.2	The system shall associate each student with a current level, stream, classroom, and academic year.
FR-2.3	The system shall record one or more parents/guardians per student with relationship, phone, email, and address.
FR-2.4	The system shall generate or accept a unique admission number per student.

3.3 Staff Management

ID	Requirement
FR-3.1	The system shall register staff with names, gender, phone, email, and staff type.
FR-3.2	The system shall associate teachers with the subjects they teach.
FR-3.3	The system shall associate teachers with the classrooms they are assigned to.

3.4 Academic Structure

ID	Requirement
FR-4.1	The system shall maintain subjects with a name, code, and associated level.
FR-4.2	The system shall maintain levels (e.g., O-Level, A-Level).
FR-4.3	The system shall maintain streams linked to a level.
FR-4.4	The system shall maintain classrooms linked to a level and stream.
FR-4.5	The system shall support A-Level subject combinations.
FR-4.6	The system shall maintain academic years with a status (active, archived).

3.5 Timetables & Lesson Plans

ID	Requirement
FR-5.1	The system shall define timetable entries mapping a subject, teacher, and class to a day and time range.
FR-5.2	The system shall allow teachers to submit lesson plans per subject, class, and academic year.
FR-5.3	The system shall support lesson plan templates and file attachments.

3.6 Assessments & Results

ID	Requirement
FR-6.1	The system shall maintain assessment types (assignment, test, quiz, midterm, final exam).
FR-6.2	The system shall record student results including marks, grade, and comment per subject and assessment.
FR-6.3	The system shall associate each result with an academic year, term, and recording teacher.
FR-6.4	The system shall generate exam reports for students.

3.7 Finance & Payments

ID	Requirement
FR-7.1	The system shall define payment terms (e.g., Term 1, Term 2, Books, Registration).
FR-7.2	The system shall define fee structures per class and payment term with an amount.
FR-7.3	The system shall record payments with amount paid, balance, status, and transaction reference.
FR-7.4	The system shall summarise expected, paid, and outstanding amounts per class.

3.8 Attendance

ID	Requirement
FR-8.1	The system shall record daily attendance per student with status (present, absent, late) and comment.
FR-8.2	The system shall support autofill of default attendance status for a class.
FR-8.3	The system shall optionally notify a parent when a student is marked absent.

3.9 Discipline, Clubs & Activities

ID	Requirement
FR-9.1	The system shall record discipline incidents with a predefined type, severity, description, action taken, date, and recorder.
FR-9.2	The system shall maintain clubs with a name and description.
FR-9.3	The system shall enrol students into clubs per academic year.
FR-9.4	The system shall record club activities with title, description, date, and recorder.

3.10 Calendar & Student Leadership

ID	Requirement
FR-10.1	The system shall maintain calendar events with title, description, date, type, and optional attachment.
FR-10.2	The system shall optionally notify parents of calendar events, including fee-payment reminders.
FR-10.3	The system shall define leadership positions (e.g., prefect, head boy, head girl).
FR-10.4	The system shall assign students to leadership positions per academic year with start and end dates.

4. Data Model

The data model is organised into master tables (reference data), transactional tables (operational records), and join tables (many-to-many relationships). Field lists reflect the current draft and will be refined with types, constraints, and indexes during expansion. All tables are dialect-neutral so a single schema definition compiles for both D1 (SQLite) and PostgreSQL.

4.1 Master Tables

Table	Key Fields
school	id, name, motto, logo, themeColors, address, phone, email
users	id, username, passwordHash, roleId, staffId (nullable), studentId (nullable)
roles	id, roleName
students	id, firstName, middleName, lastName, gender, dob, admissionNumber, dateOfAdmission, currentLevelId, currentStreamId, classroomId, academicYearId
parents	id, studentId, fullName, relationship, phone, email, address
staff	id, firstName, lastName, gender, phone, email, staffType
subjects	id, name, code, levelId
levels	id, name
streams	id, levelId, streamName
classrooms	id, name, levelId, streamId
academicYears	id, yearName, status
assessmentTypes	id, typeName

4.2 Transactional Tables

Table	Key Fields
results	id, studentId, subjectId, assessmentTypeId, academicYearId, termId, marks, grade, comment, teacherId, dateRecorded
payments	id, studentId, feeStructureId, amountPaid, balance, status, transId, datePaid
attendance	id, studentId, academicYearId, termId, date, status, comment, notifyParent
timetables	id, classId, subjectId, teacherId, dayOfWeek, startTime, endTime
lessonPlans	id, teacherId, subjectId, classId, academicYearId, attachmentUrl, templateUsed, dateSubmitted
discipline	id, studentId, typeId, severity, description, actionTaken, date, recordedBy
calendarEvents	id, title, description, academicYearId, date, attachment, notifyParent, eventType
leadershipPositions	id, title, description
studentLeadership	id, studentId, positionId, academicYearId, startDate, endDate
clubs	id, name, description
clubActivities	id, clubId, title, description, academicYearId, date, recordedBy

4.3 Join Tables

Table	Relationship & Key Fields
student_streams	Student <-> Stream: id, studentId, streamId, yearId
student_clubs	Student <-> Club: id, studentId, clubId, academicYearId
teacher_subjects	Teacher <-> Subject: id, teacherId, subjectId
teacher_classrooms	Teacher <-> Classroom: id, teacherId, classroomId

Table	Relationship & Key Fields
feeStructure	Class <-> Payment Term: id, classId, paymentTermId, amount
class_payments	Payment summary per class: id, classId, totalExpected, totalPaid, balance

5. System Architecture

5.1 Architectural Overview

YOYOL follows a layered, adapter-oriented architecture. React Router 7 serves the UI and hosts server-side loaders and actions in the same application. Business logic lives in a framework-independent service layer. All persistence passes through a database adapter interface, keeping the runtime (Cloudflare Workers or Node) and the database engine (D1 or Postgres) replaceable without touching feature code.

5.2 Layered Structure

Layer	Responsibility
Presentation (UI)	React Router 7 routes, components, and forms; SSR and progressive enhancement.
Route Handlers	Loaders (reads) and actions (writes); request validation and response shaping.
Service / Domain	Business rules and workflows, independent of framework and database.
Repository	Feature-facing data operations expressed against the adapter contract.
Database Adapter	Concrete implementation for D1 or Postgres selected by configuration.

5.3 Request Flow

1. A browser request reaches the React Router 7 server handler (Worker or Node).
2. The matching loader or action validates input and calls the service layer.
3. The service layer applies business rules and calls repositories.
4. Repositories invoke the configured database adapter, which executes against D1 or Postgres.
5. Results flow back and are server-rendered or returned as data.

5.4 Runtime Portability Note

Verified constraint: the Cloudflare Workers runtime is not Node.js. Server-side code must avoid Node-only APIs unless a portable shim exists, and Cloudflare bindings (such as D1) are injected via React Router 7's server context. This keeps a single codebase deployable to both targets.

6. Technology Stack & Database Adapter

6.1 Technology Stack

Concern	Selection	Status
Language	TypeScript (strict mode)	Confirmed
Framework (FE + BE)	React Router 7, framework mode with SSR	Confirmed
Package manager / repo	pnpm workspaces (monorepo)	Confirmed
Runtime target A	Cloudflare Workers + D1	Confirmed
Runtime target B	Node.js container + PostgreSQL	Confirmed
Database access	Drizzle ORM behind a database adapter	Confirmed
Unit / integration testing	Vitest (with @cloudflare/vitest-pool-workers)	Confirmed
End-to-end testing	Playwright	Confirmed
Containerisation	Docker + Docker Compose	Confirmed
CI/CD	GitHub Actions	Confirmed
API contract	OpenAPI 3.1	Confirmed

All selections above are confirmed internal engineering decisions for the YOYOL build.

6.2 Database Adapter Design

The database layer is defined by a single adapter contract that every backend implements. Feature code (services and repositories) depends only on this contract and on the shared Drizzle schema, never on D1 or Postgres directly. The active adapter is resolved once at startup from configuration. This satisfies NFR-6 (portability): the same application runs unchanged on either database.

6.2.1 Drizzle Driver per Target

Drizzle ORM is the single query and schema layer for both targets. The schema is written once; each runtime binds it to a different Drizzle driver. This split is required because of a verified platform constraint (see note below).

Target	Drizzle Driver	Connection
Cloudflare Workers	drizzle-orm/d1	D1 binding injected via the React Router 7 server context.
Node.js container	drizzle-orm/node-postgres	Postgres connection from DATABASE_URL.

Verified constraint: the native postgres-js/TCP driver does not run on Cloudflare Workers, because Workers cannot open raw TCP sockets to most ports; D1 is reachable only from inside a Worker. The adapter therefore selects the D1 driver on the edge and the node-postgres driver in the container. Both expose the same Drizzle query API, so repository code is identical across targets.

6.2.2 Configuration

Variable	Purpose
DB_DRIVER	Selects the adapter: 'd1' or 'postgres'.
DATABASE_URL	Postgres connection string (Postgres driver only).
D1_BINDING	Cloudflare D1 binding name (D1 driver only).

6.2.3 Adapter Contract (illustrative)

```
// Shared Drizzle schema is defined once and imported by both adapters.
export interface DatabaseAdapter {
  // Returns a Drizzle database instance bound to the active driver.
```

```
  readonly db: DrizzleDatabase;
  transaction<T>(fn: (tx: DrizzleDatabase) => Promise<T>): Promise<T>;
  migrateToLatest(): Promise<void>;
}

// Resolved once at startup from configuration:
//   DB_DRIVER=d1        -> D1Adapter        (drizzle-orm/d1)
//   DB_DRIVER=postgres  -> PostgresAdapter  (drizzle-orm/node-postgres)
```

This contract is illustrative; the requirement is a fixed, dialect-neutral interface that exposes a Drizzle instance and a transaction wrapper. The concrete shape is finalised during design.

6.2.4 Migrations

Migrations are generated once from the shared schema with drizzle-kit and applied per target. Because a Node process cannot connect directly to D1, the two targets use different application mechanisms:

- Postgres: drizzle-kit generates and applies SQL migrations against DATABASE_URL.
- D1: the same generated migrations are applied via the drizzle-kit CLI and the Cloudflare D1 HTTP API (locally through Wrangler for development).

6.2.5 Adapter Requirements

ID	Requirement
DB-1	The system shall define one database schema in Drizzle, shared by both targets.
DB-2	The system shall access persistence only through the database adapter; no feature code shall import a driver directly.
DB-3	The adapter shall be selected at startup from DB_DRIVER, supporting the values 'd1' and 'postgres'.
DB-4	The D1 adapter shall use drizzle-orm/d1 with the D1 binding from the server context.
DB-5	The Postgres adapter shall use drizzle-orm/node-postgres with DATABASE_URL.
DB-6	Migrations shall be generated once with drizzle-kit and applied to both targets from the same migration set.
DB-7	The adapter shall expose a transaction wrapper usable identically by all repositories.

6.3 Database Layer Decision (Drizzle ORM)

Drizzle ORM is the confirmed query and schema layer for YOYOL. The rationale below records why, so the decision is traceable for the team and future contributors.

- It has native, production-grade support for both required targets: Cloudflare D1 and PostgreSQL.
- Its schema-as-code model gives fully type-safe queries and pairs naturally with TDD.
- It carries no separate query-engine binary, so latency is dominated by the network round-trip, which suits Workers cold-start and low-bandwidth conditions (NFR-1).
- drizzle-kit provides cross-dialect migration generation, so one schema drives both databases (DB-6).

Alternatives considered and rejected: Prisma (heavier runtime and historically required a Data Proxy/Accelerate service on Workers) and Kysely (a capable query builder but lower-level, requiring more hand-written adapter code and no schema/migration tooling of its own). The adapter contract in 6.2 keeps the decision reversible in principle, but Drizzle is the committed choice for the build.

7. Development Methodology & Testing

7.1 Test-Driven Development

Test-Driven Development is adopted from the first commit. Every behavioural change follows the red-green-refactor cycle: write a failing test that expresses the requirement, implement the minimum code to pass it, then refactor with the test as a safety net.

7.2 Testing Framework (Specified)

The testing stack is a confirmed decision, chosen so that a single toolchain covers both runtime targets:

Purpose	Framework	Why
Unit & integration	Vitest	Fast, TypeScript-native, shares the Vite pipeline used by React Router 7.
Workers / D1 runtime tests	@cloudflare/vitest-pool-workers	Runs Vitest tests inside the actual Workers runtime (workerd) via Miniflare, with real D1 bindings and migration helpers.
End-to-end	Playwright	Browser-level tests of full user flows against the running app.

Verified detail: @cloudflare/vitest-pool-workers executes tests within workerd, provides isolated per-test storage, and exposes D1 migration helpers (readD1Migrations / applyD1Migrations). It requires Vitest 4.1 or later. This lets the same Vitest suite exercise the edge/D1 path and, with the standard Vitest runner, the Node/Postgres path, so both adapters are verified by one framework.

7.3 Test Levels

Level	Scope	Tooling
Unit	Services and domain rules in isolation.	Vitest
Integration	Repositories against a real database, run for both adapters.	Vitest (Node/Postgres) + vitest-pool-workers (D1)
Contract	Adapter conformance: one shared suite run against D1 and Postgres.	Vitest shared suite
End-to-end	User flows through the running application.	Playwright

7.4 Testing Requirements

- TR-1: Vitest shall be the unit and integration test runner; Playwright shall be the end-to-end runner.
- TR-2: The Cloudflare Workers pool (@cloudflare/vitest-pool-workers) shall be used to test the D1/Workers path inside the real runtime, applying D1 migrations in test setup.
- TR-3: Each functional requirement (FR-*) and adapter requirement (DB-*) shall have at least one automated test referencing its ID.
- TR-4: The adapter contract shall be verified by a single shared suite executed against both D1 and Postgres.
- TR-5: A minimum coverage threshold of 80% of lines and branches shall be enforced in CI.
- TR-6: CI shall block merges when any test fails or coverage falls below the threshold.
- TR-7: Bug fixes shall begin with a failing regression test that reproduces the defect.

8. Documentation Requirements

8.1 API Documentation

- DR-1: All server endpoints and data contracts shall be described in an OpenAPI 3.1 specification kept in the repository.
- DR-2: The specification shall be the single source of truth and shall be validated in CI.
- DR-3: Human-browsable API docs shall be generated from the specification.

8.2 System Documentation

- DR-4: An architecture overview shall document layers, request flow, and the adapter design.
- DR-5: Architecture Decision Records (ADRs) shall capture significant technical decisions, including the ORM choice.
- DR-6: A data dictionary shall document every table and field.
- DR-7: Deployment guides shall cover both the Cloudflare (D1) and Docker/Postgres paths.

8.3 Contributor & User Documentation

- DR-8: The repository shall include README, CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, and CHANGELOG files.
- DR-9: A local-development quickstart shall get a contributor running with one documented command sequence.
- DR-10: End-user guides shall be provided per role (Admin, Teacher, Finance, Student, Parent).

9. Coding Standards & Conventions

9.1 Language & Style

- CS-1: TypeScript strict mode shall be enabled repository-wide; no implicit any.
- CS-2: ESLint and Prettier shall enforce linting and formatting; violations fail CI.
- CS-3: A consistent, documented file and folder structure shall be followed across the monorepo.
- CS-4: Public functions and exported types shall carry doc comments where intent is non-obvious.

9.2 Version Control & Reviews

- CS-5: Commits shall follow Conventional Commits.
- CS-6: Releases shall follow Semantic Versioning; changes recorded per Keep a Changelog.
- CS-7: Work shall land via pull requests with at least one review and green CI; direct pushes to the main branch are prohibited.
- CS-8: A pre-commit hook shall run lint, format, and fast tests locally.

9.3 Quality Gates in CI

Gate	Requirement to Pass
Lint & Format	No ESLint errors; Prettier check clean.
Type Check	TypeScript compiles with no errors in strict mode.
Tests	All unit, integration, contract, and E2E tests pass.
Coverage	At or above 80% of lines and branches.
Adapter Matrix	Contract suite passes on both D1 and Postgres.
API Spec	OpenAPI document validates.
Build & Image	Application builds and the Docker image builds successfully.

10. Open-Source Governance

10.1 Licensing & Repository Hygiene

- OS-1: The project shall be released under the MIT License, with a LICENSE file at the repository root.
- OS-2: The repository shall be public on GitHub with a descriptive README and topic tags.
- OS-3: Issue and pull-request templates shall be provided.
- OS-4: A CODE_OF_CONDUCT (Contributor Covenant) shall govern community conduct.

10.2 Contribution & Release Process

- OS-5: CONTRIBUTING shall define setup, branching, commit, and review expectations.
- OS-6: Labelled 'good first issue' items shall lower the barrier for new contributors.
- OS-7: Releases shall be tagged, versioned, and accompanied by changelog entries.
- OS-8: A SECURITY policy shall define how to report vulnerabilities privately.

11. DevOps, Containerisation & Deployment

11.1 Containerisation

- DV-1: The application shall ship with a Dockerfile producing a runnable image.
- DV-2: A Docker Compose file shall bring up the app and a Postgres database for local development.
- DV-3: Configuration shall be supplied via environment variables; a documented .env.example shall list every variable.

11.2 CI/CD

- DV-4: GitHub Actions shall run the full quality-gate pipeline on every pull request.
- DV-5: The pipeline shall run the adapter contract suite against both D1 and Postgres.
- DV-6: Tagged releases shall build and publish the Docker image.

11.3 Deployment Targets

Target	Database	Notes
Cloudflare Workers	D1	Edge deployment via Wrangler; D1 bound through server context.
Docker / self-host	PostgreSQL	Single image plus Postgres; suits on-premise and VPS hosting.

Verified note: the Cloudflare CLI currently scaffolds a React Router project on the newer v8 line; this project targets React Router 7 as specified, and the migration path to v8 is tracked as an open item.

12. Security

Security is a first-class requirement because YOYOL holds personal records of minors and financial data. The controls below apply to both deployment targets and are verified in CI where automatable.

12.1 Authentication

Authentication is token-based (JWT), implemented in the shape recommended for a React Router 7 framework-mode application so the raw token is never exposed to client-side JavaScript.

ID	Requirement
SEC-1	Users shall authenticate with a username/email and password; passwords shall be stored using a strong adaptive hash (e.g. Argon2id or bcrypt) with per-user salt.
SEC-2	On login, the server shall issue a short-lived access token (JWT, 5-15 minute lifetime) and a longer-lived refresh token.
SEC-3	Both tokens shall be delivered in HttpOnly, Secure, SameSite cookies; tokens shall never be stored in localStorage or exposed to JavaScript.
SEC-4	Refresh tokens shall be rotated on every use and be revocable server-side; a server-side store shall allow immediate session invalidation (logout, compromise).
SEC-5	Loaders and actions shall verify the session server-side before returning data or performing writes; protected routes shall not rely on client-side redirects alone.
SEC-6	Failed-login attempts shall be rate-limited and, on repeated failure, temporarily locked to resist brute force.

12.2 Authorization

ID	Requirement
SEC-7	Access shall be governed by role-based access control (RBAC) using the roles in Section 2.3; every action shall check the caller's role.
SEC-8	Authorization shall be enforced on the server for every request; the UI hiding a control is never the sole protection.
SEC-9	Data scoping shall ensure a user only accesses records they are entitled to (e.g. a parent sees only their linked student; a teacher only their classes).

12.3 Application & Supply-Chain Security

ID	Requirement
SEC-10	All input shall be validated and typed at the boundary (loaders/actions) with a schema validator before use.
SEC-11	Database access shall use parameterised queries via Drizzle; no string-concatenated SQL.
SEC-12	Output shall be safely encoded to prevent XSS; framework escaping shall not be bypassed without review.
SEC-13	State-changing form submissions shall be protected against CSRF (SameSite cookies plus anti-CSRF tokens where applicable).
SEC-14	Security-relevant HTTP headers shall be set (e.g. Content-Security-Policy, HSTS, X-Content-Type-Options).
SEC-15	Secrets shall be supplied via environment/binding configuration only; no secret shall be committed to the repository.
SEC-16	Dependencies shall be scanned for known vulnerabilities in CI (e.g. audit and automated dependency updates); high-severity issues shall block release.

13. Data Protection & Privacy

YOYOL processes personal data about minors and payment information. It follows widely accepted data-protection principles (data minimisation, purpose limitation, storage limitation, integrity and confidentiality). Deployments remain responsible for compliance with the data-protection law applicable in their jurisdiction.

13.1 Data Classification

Class	Examples	Handling
Sensitive personal	Student and guardian identity, contact details, results.	Access-controlled, encrypted in transit; least-privilege.
Financial	Payment records, transaction references.	Access-controlled; retained per policy; auditable.
Operational	Timetables, calendar, clubs.	Standard access controls.
Public	School name, branding.	No special handling.

13.2 Requirements

ID	Requirement
DP-1	The system shall collect only data necessary for its stated functions (data minimisation).
DP-2	All traffic shall be encrypted in transit (TLS); no personal data over plaintext channels.
DP-3	The system shall support data-subject requests: export and deletion of a person's data where lawful.
DP-4	Data retention periods shall be configurable; records past retention shall be purgeable.
DP-5	Automated, restorable backups shall be supported for the Postgres deployment, with documented restore steps; D1 backup/restore shall follow Cloudflare's mechanisms.
DP-6	Access to sensitive data shall be logged for accountability (see Section 14).
DP-7	A breach-response procedure shall be documented, including notification steps.
DP-8	Special care shall apply to minors' data; guardian association shall gate access to a student's records.

14. Observability & Error Handling

The system must be operable and debuggable by self-hosters and by the NOTTECH team, without leaking sensitive data through logs or error messages.

14.1 Logging & Monitoring

ID	Requirement
OB-1	The application shall emit structured (JSON) logs with a consistent schema and severity levels.
OB-2	Each request shall carry a correlation/request ID propagated through logs to trace a request end-to-end.
OB-3	Logs shall never contain secrets, passwords, tokens, or full payment identifiers; sensitive fields shall be redacted.
OB-4	The application shall expose a health-check endpoint reporting service and database status.
OB-5	Key operational metrics (error rate, latency, database health) shall be observable by the operator.

14.2 Error Handling

ID	Requirement
OB-6	Errors shall follow a defined taxonomy (validation, authorization, not-found, conflict, internal) mapped to correct HTTP status codes.
OB-7	User-facing error messages shall be clear and safe; internal details and stack traces shall never be shown to end users.
OB-8	Route-level error boundaries shall render friendly fallback UI rather than a blank page.
OB-9	Unhandled errors shall be logged with context and correlation ID for diagnosis.

14.3 Audit Trail

- OB-10: Security- and finance-relevant actions (login, permission changes, payment records, record deletions) shall be recorded in an append-only audit log capturing actor, action, target, and timestamp (supports NFR auditability and DP-6).

15. API Design & Conventions

Server endpoints and data contracts follow consistent conventions so the API is predictable for the UI, for integrations, and for future contributors. The OpenAPI 3.1 specification (Section 8) is the source of truth.

ID	Requirement
API-1	Endpoints shall follow consistent, resource-oriented naming and correct HTTP semantics (safe reads, idempotent writes where applicable).
API-2	The API shall be versioned so breaking changes do not disrupt existing clients.
API-3	Error responses shall share a single consistent shape (code, message, and details) aligned with the error taxonomy (OB-6).
API-4	List endpoints shall support pagination, and filtering/sorting where relevant, with documented limits.
API-5	Request and response bodies shall be schema-validated and reflected accurately in the OpenAPI document.
API-6	Date/time values shall use a single standard (ISO 8601, UTC) across the API.
API-7	The OpenAPI document shall be validated in CI and kept in sync with the implementation.

16. Accessibility & Internationalization

16.1 Accessibility

The interface must be usable by people with disabilities and on the low-cost devices common in the target market.

ID	Requirement
AC-1	The UI shall conform to WCAG 2.2 Level AA as the target standard.
AC-2	All functionality shall be operable by keyboard; focus order and visible focus shall be correct.
AC-3	Semantic HTML and ARIA shall be used so assistive technologies can interpret the UI; forms shall have associated labels.
AC-4	Colour shall not be the sole means of conveying information; text/background contrast shall meet AA.
AC-5	Automated accessibility checks shall run in CI, with manual checks for key flows.

16.2 Internationalization & Localization

ID	Requirement
I18N-1	User-facing strings shall be externalised for translation; no hard-coded display text.
I18N-2	The system shall support at least English and Swahili, reflecting the target market.
I18N-3	Dates, numbers, and currency shall be formatted per the active locale.
I18N-4	The architecture shall allow adding further locales without code changes to features.

17. Non-Functional Requirements

These cross-cutting qualities complement the dedicated Security (12), Data Protection (13), Observability (14), and Accessibility (16) sections.

ID	Category	Requirement
NFR-1	Performance	Responsive under low-bandwidth conditions typical of the target region.
NFR-2	Usability	Mobile-first interface usable on small screens.
NFR-3	Security	Passwords stored as secure hashes; access governed by RBAC.
NFR-4	Reliability	Transactional records persisted durably and consistently.
NFR-5	Scalability	Data model supports multiple academic years and archival.
NFR-6	Portability	Runs unchanged on D1 or Postgres via configuration.
NFR-7	Maintainability	Modules independently extensible; enforced coding standards.
NFR-8	Testability	Architecture supports isolated, automated testing at every level.
NFR-9	Observability	Operable and debuggable; detailed in Section 14.
NFR-10	Accessibility	WCAG 2.2 AA target; detailed in Section 16.
NFR-11	Localisation	Supports English and Swahili; regional conventions including mobile money (Section 16).
NFR-12	Auditability	Key actions captured in an append-only audit log (Section 14.3).

18. Planned Feature Set (Roadmap)

The following features extend the source notes and are planned enhancements on the YOYOL roadmap, prioritised for delivery after the core modules.

18.1 Primary: Mobile Money Fee Collection & Automated Reconciliation

A fee-collection feature integrating regional mobile money so guardians pay school fees directly from a phone, with the system reconciling each payment against a student's fee structure automatically.

Why it fits YOYOL

- Mobile money is the dominant payment rail in the target market, making this the highest-impact addition to the existing finance module.
- It closes the loop between the payments table, fee structures, and per-class summaries already in the data model.
- Automated reconciliation and receipts reduce manual finance-office workload and errors.

Outline capabilities

- Guardian-initiated payment against an outstanding balance, with a transaction reference captured on the payment record.
- Automatic status update (paid, partial, pending) and balance recalculation on confirmation.
- Digital receipt and optional parent notification on success.
- Finance dashboard showing collected, outstanding, and reconciled totals per class and term.

18.2 Secondary Roadmap Items

Feature	Benefit
Automated report-card generation	Compiles results per term into printable/exportable report cards, extending the results module.
Multi-channel parent notifications (SMS, email, in-app)	Reliable delivery of attendance alerts, fee reminders, and calendar events across low-bandwidth contexts.
Role-based dashboards & analytics	At-a-glance views of attendance trends, fee collection, and academic performance per role.
Offline-tolerant attendance capture	Lets teachers mark attendance with intermittent connectivity and sync later.
Bulk import/export (CSV)	Speeds onboarding of students, staff, and fee structures for new schools.
Audit trail & activity log	Records who changed what and when, supporting NFR-12 auditability.

19. Open Items for Expansion

Recognised gaps to be addressed in later revisions:

6. Full field-level data types, constraints, and referential integrity rules.
7. A terms table referenced by results and attendance (termId currently unresolved).
8. A-Level combinations table structure.
9. Final concrete adapter interface signature (Drizzle instance shape) settled in design.
10. Detailed grading scales and report-card layouts.
11. Mobile money provider selection and reconciliation rules.
12. Notification channels, templates, and delivery guarantees.
13. A full requirements-traceability matrix linking each requirement ID (FR, DB, SEC, DP, OB, API, AC, I18N) to its tests.
14. Chosen validation, logging, and i18n libraries fixed in design.
15. React Router 7 to 8 migration path.

End of Draft — Version 0.4